UNIVERSITÉ CAEN NORMANDIE

UFR des Sciences

Université de Caen Normandie
Département mathématiques-informatique
2024–2025
Bruno Zanuttini

L3 informatique

TP. Programmation par contraintes

Unité YSINL5D2 : Aide à la décision et intelligence artificielle

---

L'objectif de ce TP est d'écrire des classes permettant de résoudre des problèmes de satisfaction de contraintes. Toutes les classes créées pour ce faire devront l'être dans un *package* Java nommé `cp`.

Il s'agit essentiellement d'implémenter des solveurs de contraintes. Pour cela, on écrira des classes (par exemple `BacktrackSolver`) dont chaque instance sera dédiée à la résolution d'un couple (variables, contraintes). Ainsi, la méthode de résolution n'aura plus à prendre le problème en argument (ce sont les attributs de la classe qui seront utilisés). Cette méthode sera censée retourner `null` si le problème n'a pas de solution.

Cet énoncé suppose que l'interface `Constraint` et la classe `Variable` du *package* `modelling` sont implémentées (cf. 1er TP).

Tests fournis

Des tests unitaires sont fournis dans le package `cptests`. Un exemple complet d'utilisation de ces tests est :

```
import cptests.AbstractSolverTests;
import cptests.ArcConsistencyTests;
import cptests.BacktrackSolverTests;
import cptests.DomainSizeVariableHeuristicTests;
import cptests.HeuristicMACSolverTests;
import cptests.MACSolverTests;
import cptests.NbConstraintsVariableHeuristicTests;
import cptests.RandomValueHeuristicTests;
[...]
boolean ok = true;
ok = ok && AbstractSolverTests.testIsConsistent();
ok = ok && BacktrackSolverTests.testSolve();
ok = ok && ArcConsistencyTests.testEnforceNodeConsistency();
ok = ok && ArcConsistencyTests.testRevise();
ok = ok && ArcConsistencyTests.testAC1();
ok = ok && MACSolverTests.testSolve();
ok = ok && HeuristicMACSolverTests.testSolve();
ok = ok && NbConstraintsVariableHeuristicTests.testBest();
ok = ok && DomainSizeVariableHeuristicTests.testBest();
ok = ok && RandomValueHeuristicTests.testOrdering();
System.out.println(ok ? "All tests OK" : "At least one test KO");
```

Il est aussi possible de tester son propre solveur (implémentant l'interface `Solver`), en appelant :

```
ok = ok && new SolverTests(
    (vars, constraints) -> new MonSolveur(vars, constraints [...])
).testSolve();
```

1 Backtrack

Exercice 1. *Créer une interface nommée* `Solver`*, déclarant une méthode* `solve()` *qui retourne une solution (de type* `Map<Variable, Object>`*), ou* `null` *s'il n'y en a pas (le cosntructeur des classes concrètes prendra les variables et les contraintes en arguments).*

Exercice 2. *Créer une classe abstraite nommée* `AbstractSolver`*, implémentant cette interface, et possédant un constructeur qui prend un ensemble de variables (de type* `Set<Variable>`*) et un ensemble de contraintes (de type* `Set<Constraint>`*) en arguments.*

Exercice 3. *Ajouter à la classe* `AbstractSolver` *une méthode (concrète)* `isConsistent(Map<Variable, Object>)`*, qui prend en argument une affectation partielle des variables, et retourne un booléen indiquant si cette affectation satisfait toutes les contraintes qui portent sur des variables instanciées dans l'affectation (cohérence locale).*

Exercice 4. *Créer une classe nommée* `BacktrackSolver` *héritant de la classe* `AbstractSolver`*, et dont le constructeur prend un ensemble de variables et un ensemble de contraintes en arguments. Pour la définition de la méthode* `solve`*, on pourra faire appel à une seconde méthode, récursive, qui prend en argument une solution partielle et une liste de variables non instanciées, et retourne elle-même une solution étendant la solution partielle, ou* `null` *s'il n'y en a pas.*

2 Cohérence d'arc et MAC

Dans les exercices qui suivent, on ne s'intéressera qu'à des contraintes unaires et binaires (portant sur une ou deux variables). Pour tester cela, et accéder aux variables, on pourra utiliser un code inspiré du code suivant :

```
Set<Variable> scope = constraint.getScope();
if (scope.size()==1) {
  Variable var = scope.iterator().next();
  [...]
} else if (scope.size()==2) {
  Iterator<Variable> it = scope.iterator();
  Variable var1=it.next();
  Variable var2=it.next();
  [...]
} else {
  throw new IllegalArgumentException("Ni unaire ni binaire");
}
```

Note Pour les méthodes devant supprimer des valeurs en place, ne pas supprimer dans le domaine au fur et à mesure (au risque d'obtenir des `ConcurrentModificationException`), mais stocker temporairement les valeurs à supprimer dans un autre ensemble que le domaine, et les supprimer seulement à la fin du traitement.

Exercice 5. *Créer une classe nommée* `ArcConsistency`*, dont le constructeur prend un ensemble de contraintes en argument et lance une* `IllegalArgumentException` *si les contraintes ne sont pas toutes unaires ou binaires.*

Exercice 6. *Ajouter à la classe* `ArcConsistency` *une méthode nommée* `enforceNodeConsistency`*, prenant en argument un ensemble de domaines (de type* `Map<Variable, Set<Object»`*). La méthode doit supprimer, en place, les valeurs v des domaines pour lesquelles il existe une contrainte unaire non satisfaite par v. Elle doit par ailleurs retourner* `false` *si au moins un domaine a été vidé (*`true` *sinon).*

Exercice 7. *Toujours dans la classe* `ArcConsistency`*, ajouter une méthode nommée* `revise`*, prenant en arguments, dans cet ordre : une variable v_1, son domaine D_1, une autre variable v_2, et son domaine D_2. La méthode doit supprimer, en place, toutes les valeurs v_1 de D_1 pour lesquels il n'existe aucune valeur v_2 de D_2 supportant v_1 pour toutes les contraintes portant sur v_1 et v_2. La méthode doit en outre retourner* `true` *si au moins une valeur a été supprimée de D_1 (*`false` *sinon).*

Exercice 8. *Toujours dans la classe* `ArcConsistency`*, ajouter une méthode nommée* `ac1`*, qui prend en argument un ensemble de domaines (de type* `Map<Variable, Set<Object»`*), filtre tous les domaines en place en utilisant* `ac1`*, jusqu'à stabilité, et retourne* `false` *si au moins un domaine a été vidé (*`true` *sinon).*

Exercice 9. *Écrire une classe nommée* `MACSolver`*, héritant de la classe* `AbstractSolver`*, dont le constructeur prend un ensemble de variables et un ensemble de contraintes en arguments. Pour l'implémentation de la méthode* `solve`*, utiliser la classe* `ArcConsistency` *dans l'exploration de l'arbre de recherche.*

Pour la cohérence d'arc, attention à bien traiter le cas des contraintes unaires également, c'est-à-dire des contraintes n'ayant qu'une variable dans leur *scope* (en particulier, la méthode `ac1` doit assurer également la cohérence de nœud).

3 Heuristiques

Exercice 10. *Écrire une interface nommée* `VariableHeuristic`*, déclarant une méthode* `best` *qui prend en arguments, dans cet ordre, un ensemble de variables (de type* `Set<Variable>`*) et un ensemble de domaines (de type* `Map<Variable, Set<Object>>`*), et qui retourne une variable (la meilleure au sens de l'heuristique).*

Exercice 11. *Écrire une interface nommée* `ValueHeuristic`*, déclarant une méthode* `ordering` *qui prend en arguments, dans cet ordre, une variable (de type* `Variable`*) et son domaine (de type* `Set<Object>`*), et qui retourne une liste contenant les valeurs du domaine, ordonnées selon l'heuristique (la meilleure en premier).*

Exercice 12. *Écrire une classe nommée* `NbConstraintsVariableHeuristic` *implémentant l'interface* `VariableHeuristic`*, dont le constructeur prend en arguments, dans cet ordre, un ensemble de contraintes et un booléen. Le booléen indiquera si l'on préfère les variables apparaissant dans le plus de contraintes (*`true`*) ou dans le moins de contraintes (*`false`*).*

Exercice 13. *Écrire de même une classe nommée* `DomainSizeVariableHeuristic`*, ne prenant en argument qu'un booléen, indiquant ici si l'on préfère les variables avec le plus grand domaine (*`true`*) ou avec le plus petit domaine (*`false`*).*

Exercice 14. *Écrire une classe nommée* `RandomValueHeuristic`*, implémentant l'interface* `ValueHeuristic`*, dont le constructeur prend en argument un générateur aléatoire (de type* `Random`*), et dont la méthode* `ordering` *retourne une liste mélangée (pseudo-)uniformément.*

Exercice 15. *Écrire une classe nommée* `HeuristicMACSolver`*, étendant la classe* `AbstractSolver`*, et dont le constructeur prend en arguments, dans cet ordre, un ensemble de variables, un ensemble de contraintes, une heuristique sur les variables, et une heuristique sur les valeurs.*